

AirWave: A Multimodal Context-Aware Music Interaction System

Annique Aarts
UCLA
Los Angeles, CA
anniqueaarts@ucla.edu

Leah George
UCLA
Los Angeles, CA
leahgeorge23@ucla.edu

Matthew Mansourian
UCLA
Los Angeles, CA
mattmansourian@g.ucla.edu

Andrew Wan
UCLA
Los Angeles, CA
drewkeithwan@ucla.edu

Abstract- AirWave is a multimodal music interaction system designed to make music control more seamless, intuitive, and context-aware during everyday activities. The project was motivated by the limitations of traditional playback methods, especially phones and voice assistants, which can interrupt workflow, struggle in noisy settings, and fail to adapt to a user's movement within a space. To address this, AirWave combines several input and sensing modalities into one platform: IMU-based gesture recognition, voice command processing, camera-based distance tracking for automatic volume adjustment, mood detection, and integrated Spotify playback with LED feedback. The system was developed collaboratively through subsystem design, iterative testing, debugging, and final integration, with each team member contributing to both specialized modules and overall system refinement. The results showed that AirWave successfully demonstrated the feasibility of combining gesture control, speech interaction, proximity-based volume adaptation, and contextual feedback into a single working prototype. The final system provided users with a more natural way to interact with music without depending on constant manual input, showing that multimodal interaction can improve usability compared to single-modality systems. At the same time, the project also revealed practical limitations, including the complexity of coordinating multiple real-time subsystems, reliability challenges in recognition and tracking, and the need for further refinement to improve robustness. Overall, AirWave concludes that integrated multimodal control is a promising direction for future music interaction systems, offering a more adaptive and user-centered alternative to conventional playback interfaces while also opening opportunities for future improvements in reliability, accessibility, and real-world deployment.

I. INTRODUCTION

Music is part of people's daily routines, whether they are studying, cooking, exercising, or cleaning. Although technology has improved music streaming services and voice assistants, interacting with playback still disrupts the experience of listening to music. Manual device control and voice-based assistants are currently the dominant models in modern music interaction systems, though they are often problematic when engaging in activities that require constant movement. Accessing a phone mid-activity can interrupt focus and workflow. Even more advanced voice assistants tend to do poorly in noisy settings and can be

disruptive and socially awkward in public settings. Furthermore, if individuals are listening to music while moving around a room, changing the volume each time they move away from the speaker is inefficient and disruptive. Today, there is a growing need for interactive technologies that are aware of the user's context and not limited to explicit user commands. Therefore, the AirWave was developed to demonstrate that integrated gesture recognition, voice control, proximity-based volume adaptation, and mood detection into a single platform can create a more intuitive and context-aware music interaction experience than traditional single-modality systems.

Smart speakers and streaming devices have become ubiquitous in homes, dorm rooms, and workplaces, popularizing voice control and interaction while raising user demands and expectations for smooth and intuitive control. Listening to music is no longer confined to a stationary device and now follows users throughout their daily activities. Accessibility considerations also show that not all users can rely solely on speech-based interaction. These trends have contributed to growing interest in context-aware and multimodal interaction techniques using gesture, speech, and vision sensing. Instead of using a single channel of interaction, modern interactive systems should aim to automatically sense and react to users' presence and movements.

Smartphone interaction is based on physical proximity to the device, while voice assistants are based on microphones and connected systems with speech recognition. Although these are effective interaction methods, they are generally unimodal and do not use spatial awareness and user movements as input. Moving beyond these interaction methods adds a layer of complexity. Motion sensors, such as IMUs, filter data streams to identify gestures from normal movements. Speech recognition in real-time requires processing of a speech signal and associated systems for speech recognition. Vision sensing uses a camera to detect a user's face and calculate their distance from a speaker, allowing for smooth adjustment of volume according to a user's position. Together, these sensing and processing

techniques illustrate both the potential and the technical demands of developing a cohesive multimodal system capable of real-time, context-aware music interaction.

The contributions and team roles are defined as follows. Annique developed the voice recognition and command subsystem, and implemented light dancing and LED feedback, and contributed to testing. Leah implemented the IMU-based gesture recognition system, managed Spotify audio playback integration, developed the launcher.py script, and contributed to overall system integration. Matthew and Andrew worked together on the distance-based volume control system and camera tracking. Matthew was also responsible for bluetooth integration and mood detection while Andrew additionally contributed to system integration and testing. All team members contributed to design, system implementation, debugging, and final integration. For the final paper, Annique was responsible for the Introduction, Societal and Ethical Implications, and References. Andrew completed the Abstract and Conclusion. Leah was responsible for State of the Art while Matthew completed Methods and Results. Through iterative refinement and coordinated development, the team produced AirWave, a fully integrated multimodal music control system.

II. STATE OF THE ART

AirWave is an interactive speaker system that combines several existing technical methods from human-computer interaction, embedded systems, and smart media control. The project integrates Bluetooth IMU gesture recognition, offline voice recognition, emotion detection, face tracking, and networked device communication into a single user-facing product. These methods are important because they support the team's goal of creating a robust, flexible, and highly personalizable music system. Reviewing them here helps explain how AirWave works technically and how its design compares with existing approaches in related systems. The system architecture can be seen in Fig. 1.

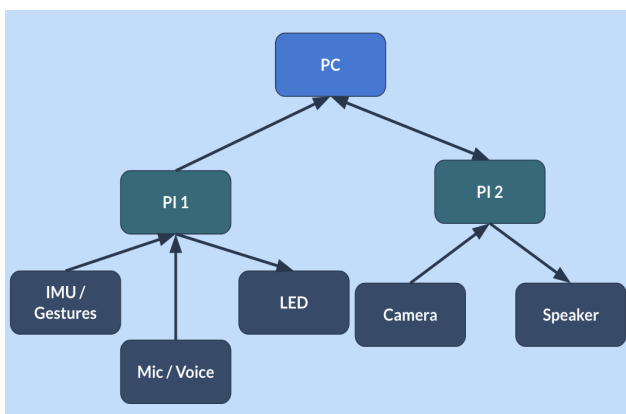


Fig. 1. AirWave System Architecture.

One of the primary methods used in AirWave is IMU-based gesture recognition through the WT901 sensor. Motion and orientation data are continuously collected when the IMU is connected to Raspberry Pi 1 via Bluetooth. Using calibration scripts, gesture data were recorded for multiple starting positions, allowing threshold values to be tuned for classification. Once identified, gestures are translated into playback commands and published to the MQTT broker. AirWave uses a double flick to enter command mode, twist gestures to move to the next or previous track, and swipe gestures to pause or resume playback. Compared with camera-based gesture recognition or wearable devices such as smartwatches and fitness trackers, this method is low-cost, fast, and does not depend on a camera view. It is also more expressive than many wearable consumer products, which often rely on simpler tapping motions rather than full gesture recognition. Although accidental triggers can still occur, the method proved relatively accurate overall.

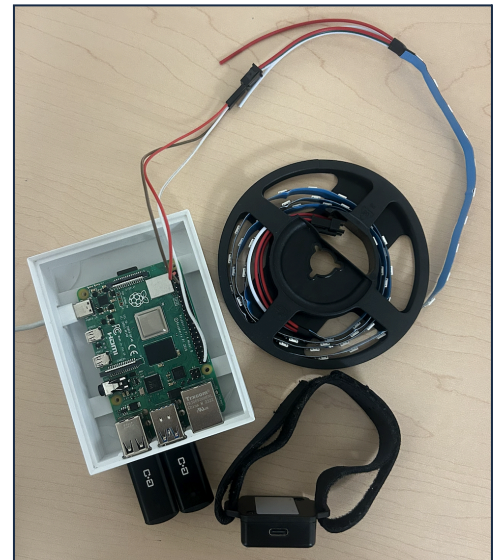


Fig 2. Raspberry Pi 1 - Voice and Gesture Recognition

AirWave also uses microphone-based voice command recognition to support natural and accessible interaction. Spoken words such as play, pause, next, previous, and volume adjustments are detected and mapped to playback commands using the offline Vosk API. Through repeated rounds of testing, thresholds were adjusted to reduce false positives caused by the system's own music playback. Compared with smart-speaker ecosystems such as Alexa or larger cloud-based speech recognition platforms, this offline approach offers privacy and direct local control while still complementing the gesture system. Its main tradeoff is that increasing thresholds to avoid false positives can also produce false negatives, meaning users may occasionally need to repeat commands more clearly.

A third major method is camera-based face tracking with a motorized pan-tilt mount. Live video is processed in real

time using OpenCV cascade classifiers to detect and lock onto the user's face, while servo motors continuously adjust the camera angle to keep the user centered in the frame. In addition to user tracking, the system estimates distance through changes in the face bounding-box area and uses this information to trigger automatic volume adjustments. This creates a personalized and adaptive interaction style that is not typical of most current smart speaker systems. Compared with fixed-camera setups or more advanced computer vision systems, AirWave's method provides effective tracking with modest hardware, although processing delays can slightly slow the volume response and performance decreases under poor lighting conditions.

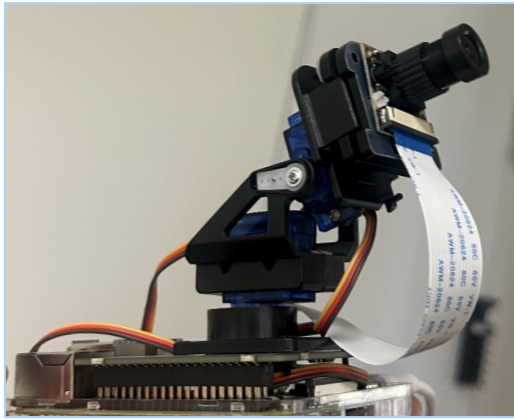


Fig 3. Camera and Pan-Tilt Tracking Module.

AirWave further extends personalization through facial-expression-based mood detection. The system periodically analyzes the user's expression every 15 seconds while tracking is active. Using OpenCV, it measures smile presence, eye count, brightness, contrast, and skin warmth; these are weighted to classify the user's mood as happy, sad, or calm. A matching playlist recommendation is then pushed directly to the dashboard. Compared with recommendation systems based primarily on listening history, this method introduces an emotion-aware media experience that is uncommon in mainstream smart audio products. Its value lies in increased personalization and adaptive playback, though the classification is not perfectly accurate and can misidentify the user's mood.

Another core method is AirWave's distributed embedded architecture, which uses a Mac computer and two Raspberry Pis coordinated through MQTT and WebSockets. The Mac hosts the launcher, dashboard, and broker; Pi 1 handles gesture recognition, voice recognition, and LED feedback; and Pi 2 handles camera-based tracking and speaker output. This division of labor allows modularity, real-time coordination, and visible user customization through the dashboard. Compared with single-device or non-networked systems, the MQTT-based design makes AirWave more flexible and interactive, though it also introduces dependence on network reliability and currently works only

with Mac computers. In addition, AirWave handles audio playback through Spotify for Developers using the Spotify Web API. During setup, the user connects their Spotify account through the launcher, after which playback commands such as play, pause, next, previous, and playlist support are issued through the API to the selected Bluetooth-connected output device. This method allows direct control of a real music account without requiring a custom playback engine. Compared with local file playback, custom media backends, or commercial assistant ecosystems, Spotify integration offers a familiar and reliable platform, but it is limited by Spotify's platform rules and currently requires Spotify Premium.

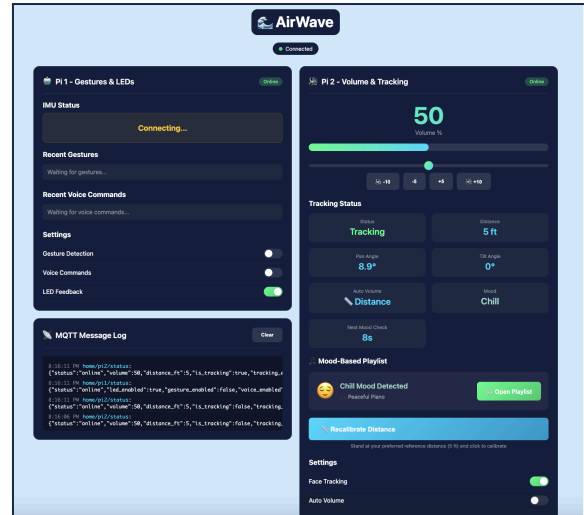


Fig. 4. AirWave User Dashboard.

The experimental methods used in AirWave focused on real hardware testing rather than simulation. Because the project depends heavily on hardware interaction, the team relied on unit functional testing, software integration testing, and user interaction testing. Individual subsystems such as gesture detection, MQTT communication, voice recognition, and face tracking were first tested independently. Metrics such as false positives, false negatives, response timing, connection reliability, and usability were measured and repeatedly adjusted until acceptable performance was reached. After subsystem validation, integrated testing was performed by starting Mosquitto and the launcher, connecting the Mac and both Raspberry Pis to the same network, verifying dashboard and MQTT communication, and then testing gestures, voice commands, face tracking, automatic volume adjustment, and mood-based recommendations in sequence. Errors, delays, and failures were recorded, and trials were repeated across different users and devices to evaluate deployment readiness.

A significant part of the project involved adapting existing code and hardware to new uses. Existing software libraries such as Vosk, OpenCV, and MQTT communication libraries were originally developed for speech recognition, computer vision, and inter-device messaging, but were adapted in

AirWave to enable voice-controlled playback, face tracking, mood detection, and synchronized communication between the Mac and both Raspberry Pis. Reusing these frameworks reduced development time, improved reliability, and simplified debugging. Similarly, general-purpose hardware components such as Raspberry Pis, an IMU sensor, a microphone, a camera module, a servo mount, an LED strip, and a speaker were repurposed into a unified music-control system. Pi 1 was adapted for gesture and voice sensing, while Pi 2 was adapted for face tracking and audio output. The IMU became a wearable gesture controller, the camera and servos became a user-tracking mechanism, and the LED strip became a feedback system for recognized commands. This reuse of existing hardware made AirWave low-cost and accessible while enabling rapid prototyping, though it also introduced constraints such as Bluetooth range limits, lighting sensitivity, network dependence, and the quality limitations of consumer-grade components.

Overall, the state of the art behind AirWave shows that the system draws on well-established methods in gesture recognition, voice recognition, computer vision, affective computing, and IoT communication, but combines them in a distinctive way for hands-free and personalized music control. The project's experimental testing demonstrated that these methods can be integrated successfully in a real hardware environment, while the adaptation of existing code and hardware shows that a feasible and innovative smart audio system can be built using accessible tools and components.

III. METHODS AND RESULTS

The development of AirWave followed a modular and iterative approach, where each subsystem was implemented and tested independently before full system integration. This strategy allowed the team to isolate technical issues early, verify subsystem functionality, and reduce the risk of one component interfering with another during development. The overall system architecture consisted of two Raspberry Pis and a host computer acting as an MQTT broker, allowing different modules to communicate in real time through message passing rather than direct coupling. Raspberry Pi 1 was responsible for gesture recognition, voice recognition, and LED feedback, while Raspberry Pi 2 handled camera-based tracking, mood detection, distance-based volume control, and speaker interaction. The host computer managed the dashboard, launcher, and broker services. This distributed design reduced interference between subsystems, improved modularity, and supported scalable integration.

At a system level, AirWave functioned as a coordinated sensing and control pipeline. User input entered the system through the IMU bracelet, spoken commands, or camera-based observation, depending on the interaction type. These inputs were processed locally on the appropriate Raspberry Pi and then communicated through MQTT so

that playback, dashboard updates, LED feedback, and adaptive behaviors could be synchronized without tightly coupling all functions into a single device. This workflow allowed multiple forms of interaction to operate at the same time while still maintaining a unified user experience.

The gesture recognition subsystem used a wearable IMU bracelet connected to Raspberry Pi 1 via Bluetooth. Motion and orientation data from the sensor were continuously streamed and analyzed to detect specific gestures such as play, pause, next track, and previous track. To reduce accidental activation during normal hand movement, the system introduced a command mode triggered by a double-flick gesture before playback commands could be accepted. Gesture thresholds were refined using calibration data collected from multiple users and from different starting positions to improve robustness across variations in motion. During testing, false triggers and sensitivity issues were observed, particularly when users made sudden but unintended movements. These issues were mitigated by refining threshold values, introducing activation delays, and requiring explicit command mode entry before gestures were interpreted as playback commands.

Gesture testing focused on both recognition reliability and usability. Users repeated the same gesture sequences from different body positions and at different speeds so that the team could observe how consistently the system distinguished intended commands from natural motion. For quantitative evaluation, recognition accuracy was estimated from repeated sets of gesture attempts, while false trigger rate was estimated from normal-motion trials in which users moved naturally without intending to issue a command. Response time was measured as the delay between performing a gesture and the resulting playback action. This process showed that gesture recognition became much more stable once command mode was introduced, since it separated intentional input from ordinary arm movement during everyday activity.

TABLE I
GESTURE RECOGNITION PERFORMANCE ACROSS REPEATED TRIALS

Variable	Trial 1	Trial 2	Trial 3	Avg
Recognition Accuracy (%)	17/20 gestures correct (85%)	17/20 gestures correct (85%)	19/20 gestures correct (95%)	88.3%
False Trigger Rate (%)	2/20 normal-motion trials triggered unintentionally (10%)	3/20 normal-motion trials triggered unintentionally (15%)	1/20 normal-motion trials triggered unintentionally (5%)	10%
Response Time (ms)	474 ms	452 ms	431 ms	452 ms

The voice command subsystem was implemented on Raspberry Pi 1 using a microphone and offline speech recognition. Spoken commands such as play, pause, next, previous, and volume adjustments were processed locally and then published to the MQTT broker for execution. Using an offline pipeline helped preserve privacy and reduced dependence on internet connectivity, while also making the system more responsive in local operation. Testing involved repeating commands across different users, voice pitches, speaking speeds, and environmental conditions to evaluate command accuracy, latency, and false-trigger rate. One major issue encountered was microphone sensitivity, where the music produced by the system itself could sometimes be misinterpreted as a valid command. This issue was reduced by lowering microphone sensitivity, adjusting recognition thresholds, and introducing clearer prompts for when users should speak. Although the subsystem worked well in controlled conditions, performance was less reliable in noisier environments.

Voice testing highlighted the tradeoff between responsiveness and selectivity. Increasing sensitivity made the system more likely to recognize quiet or soft-spoken commands, but it also increased the chance that background music or ambient noise would be misclassified as speech. Lowering sensitivity reduced those false triggers, but sometimes required users to speak more clearly or repeat commands. As a result, the team tuned the subsystem to favor reliable command recognition over maximum sensitivity, since false activations were more disruptive to the overall experience than occasional missed commands. Accuracy in quiet and noisy environments was estimated from repeated command trials, while latency was measured as the delay between the user speaking and the system carrying out the command.

TABLE II
VOICE RECOGNITION PERFORMANCE ACROSS THREE
REPEATED TRIALS

Variable	Trial 1	Trial 2	Trial 3	Avg
Accuracy (%)	16/20 spoken commands recognized correctly (80%)	18/20 spoken commands recognized correctly (90%)	18/20 spoken commands recognized correctly (90%)	86.6%
Accuracy in Noise (%)	12/20 spoken commands recognized correctly (60%)	14/20 spoken commands recognized correctly (70%)	16/20 spoken commands recognized correctly (80%)	70%
Latency (ms)	587 ms	620 ms	600 ms	602 ms

The camera tracking and distance-based volume control subsystem was implemented on Raspberry Pi 2 using OpenCV and a motorized pan-tilt mount. Real-time video processing was used to detect and track the user's face, and the size of the face bounding box was used as an approximate indicator of distance from the system. This distance estimate was then used to adjust speaker volume automatically, while the camera mount continuously repositioned to keep the user centered in frame. In addition to tracking, the camera subsystem supported periodic mood analysis by measuring facial-expression-related features such as smile presence, eye visibility, brightness, contrast, and color warmth. Testing involved having users move at different speeds and distances to evaluate tracking stability, positional accuracy, and the smoothness of the automatic volume response. Initial implementations showed jitter, overshoot, and latency in both tracking and volume adjustment, but these were reduced through smoothing filters, tuning of control parameters, and recalibration of the face-area reference distance. However, some latency remained during rapid movement or when lighting conditions were poor.

This subsystem required especially careful tuning because it combined sensing, estimation, and physical motion. The face bounding-box area provided a practical low-cost way to estimate relative distance, but it was sensitive to lighting, head angle, and temporary tracking loss. To make the automatic response feel smoother, the team applied filtering to both the tracking position and the volume output so that small fluctuations would not cause constant camera jitter or abrupt volume changes. Recalibration functions were also added so that reference distance could be reset when needed. For evaluation, tracking accuracy was estimated from repeated video sessions in which the user moved around the room while the system attempted to maintain alignment. Volume-control accuracy was estimated from movement trials in which the expected volume response could be compared to the system's observed output. These changes improved stability, but testing showed that fast movement and inconsistent lighting still affected performance more than in the other subsystems.

TABLE III
CAMERA TRACKING AND DISTANCE-BASED
VOLUME-CONTROL PERFORMANCE ACROSS THREE REPEATED
TRIALS

Variable	Trial 1	Trial 2	Trial 3	Avg
Tracking Accuracy (%)	17/20 video sessions maintained correct tracking (85%)	18/20 video sessions maintained correct tracking (90%)	18/20 video sessions maintained correct tracking (90%)	88.3%
Tracking Latency (ms)	721 ms	701 ms	683 ms	701 ms
Volume Control Accuracy (%)	16/20 movement trials produced expected volume response (80%)	14/20 movement trials produced expected volume response (70%)	17/20 movement trials produced expected volume response (85%)	78.3%

The system also included LED feedback and audio integration. A WS2812B LED strip connected to Raspberry Pi 1 provided immediate visual confirmation of recognized commands and also implemented music-reactive “light dancing” synchronized with playback. This feedback helped make the system state more visible to users, especially when using gesture or voice commands from across the room. Spotify integration allowed users to stream music via Bluetooth and control playback through the system interface using the Spotify Web API. This allowed AirWave to interact with a real streaming platform rather than relying on local audio files, making the prototype more representative of a practical consumer-facing system.

After subsystem-level testing, full integration was performed. The MQTT-based communication system enabled asynchronous operation, where gesture, voice, camera, and audio modules ran in parallel and communicated through published messages. This architecture made it possible for one subsystem to send commands without needing direct control over another subsystem’s internal implementation. Integrated system testing evaluated end-to-end latency, system stability, and overall user experience across real-world scenarios such as moving around a room, multitasking, or switching rapidly between different forms of input. These tests were important because multimodal systems must coordinate multiple streams of sensing and response at the same time.



Figure 5. AirWave prototype

Integrated testing was conducted through repeated real-world usage sessions in which users interacted with AirWave while performing normal room-scale activities, such as walking between different positions, speaking commands while music was playing, and alternating between gesture and voice inputs. During these trials, the team observed whether commands were recognized correctly, whether the camera remained locked onto the user, how smoothly the automatic volume adjustment responded to movement, and whether the dashboard reflected system state changes in real time. Particular attention was given to cases where multiple subsystems were active at once, since these situations placed the greatest demand on synchronization and system responsiveness. The team also tested recovery behavior, such as reconnecting devices, recalibrating distance estimates, and restarting services after communication failures. These repeated trials helped verify that AirWave could operate as a unified interactive system rather than as a collection of isolated features.

For integrated evaluation, feedback delay was measured as the time between a recognized input and visible LED confirmation, end-to-end latency was measured as the total delay between user action and completed system response, and system stability was estimated from repeated full-system runs completed without major failure. Together, these metrics helped evaluate not only the performance of the individual subsystems, but also how well they functioned when operating simultaneously.

TABLE IV
INTEGRATED SYSTEM PERFORMANCE ACROSS THREE
REPEATED TRIALS

Variable	Trial 1	Trial 2	Trial 3	Avg
Feedback Delay (ms)	167 ms	151 ms	148 ms	155 ms
End-to-End Latency (ms)	877 ms	861 ms	823 ms	853 ms
System Stability	18/20 integrated test runs completed without major failure (90%)	18/20 integrated test runs completed without major failure (90%)	19/20 integrated test runs completed without major failure (95%)	91.6%

In terms of results, gesture recognition performed reliably after calibration, with reduced false-trigger rates due to the introduction of command mode and threshold tuning. Voice recognition worked well in controlled environments but showed reduced accuracy in noisy conditions, confirming expected limitations of speech-based interaction in realistic listening spaces. Camera tracking successfully maintained user alignment and enabled smooth volume adjustment in most cases, though tracking latency was observed during fast motion and occasional tracking errors occurred under less favorable lighting conditions. Mood detection and playlist recommendation demonstrated the feasibility of adding contextual personalization, though this feature remained less consistent than the primary control functions. LED feedback enhanced usability by providing immediate confirmation of recognized inputs, and overall system testing demonstrated stable operation over extended use.

Team collaboration played a significant role in achieving these results. Because each subsystem was developed by different team members, integration required constant communication and coordination to ensure compatibility between modules. Gesture and voice commands developed on Raspberry Pi 1 had to be transmitted through the MQTT broker and interpreted by Raspberry Pi 2 for playback control. Similarly, the camera tracking and volume control system required alignment with gesture overrides and manual controls, which led to multiple debugging sessions where team members worked together to resolve priority conflicts and inconsistent state behavior. Group testing also helped identify edge cases that were not visible during individual development, such as timing issues, recognition overlap, and subsystem interference. This collaborative process was essential in refining the system and ensuring that all components functioned together as a cohesive multimodal platform.

Another important observation from this project was how system performance changed when transitioning from isolated subsystem testing to full system integration. While

individual components performed well independently, combining them introduced challenges related to timing, resource allocation, and synchronization. Running real-time video processing alongside continuous IMU data streaming, voice monitoring, LED control, and audio playback increased overall system load, occasionally leading to small delays in response. This highlighted the importance of efficient communication, concurrency, and subsystem prioritization in embedded multimodal systems. Although MQTT and parallel execution helped mitigate many of these issues, the system still demonstrated that multimodal interaction systems are inherently more complex than single-modality designs.

Despite these limitations, the results demonstrate that AirWave successfully achieved a functional multimodal music interaction system. The integration of gesture recognition, voice control, camera tracking, mood-based contextual feedback, and adaptive volume control created a cohesive and intuitive user experience. While further optimization is needed to improve reliability, reduce latency, and strengthen robustness across different environments, the system effectively demonstrates the potential of context-aware, multimodal interaction for future music control systems.

IV. SOCIETAL AND ETHICAL IMPLICATIONS

AirWave was designed to make music interaction easier and more responsive to the user's situation, but it also raises important societal and ethical questions. Since the system relies on continuous sensing through a camera, microphone, and gesture input, it collects information that many users would consider sensitive, especially in a home environment. Even though AirWave is designed to run locally and avoid constant cloud-based transmission, it still interprets facial expressions, movement patterns, and voice commands in real time. Video and audio are not meant to be stored for extended periods, but the fact that the camera and microphone remain active during operation can still create concerns about privacy, consent, and user awareness. In practice, users need to clearly understand when the system is sensing, what information it uses, and how that information is handled.

Another important ethical issue is control. AirWave is meant to feel helpful and adaptive, but systems that make automatic decisions can easily become frustrating if they behave in ways the user did not intend. In this project, the system can automatically adjust volume based on distance, respond to gesture and voice commands, and recommend music based on detected facial expressions. While these features are intended to improve convenience and personalization, they also introduce the risk that the system may misinterpret the user or respond in ways that feel disruptive rather than useful. If errors build up over time, even well-intentioned adaptive behavior can reduce trust in the system.

To address these concerns, AirWave was designed so that the user remains in control. Manual user input always takes priority over automated behavior, and adaptive features can be turned off when needed. These controls allow users to immediately override the system if its responses become inconvenient, and they also provide a simple way to disable sensing-based features entirely if there are privacy concerns. This was an important part of the design philosophy: the system should support the user without taking control away from them.

Overall, AirWave shows that context-aware and multimodal systems can be developed in a way that is both useful and ethically thoughtful, but only if privacy, transparency, and user control are treated as core design requirements rather than afterthoughts. The project highlights that responsible interactive systems should not only work well technically, but should also make users feel informed, respected, and in charge of their own experience.

V. CONCLUSION

The most important result of this project was that AirWave successfully demonstrated a working multimodal music interaction system that combined gesture recognition, voice commands, camera-based distance tracking, mood detection, Spotify integration, and LED feedback into one platform. This is significant because it showed that music control can move beyond traditional phone input or voice-only assistants and become more natural, adaptive, and responsive to a user's environment. By integrating multiple forms of interaction, the system provided a more intuitive listening experience and highlighted the value of context-aware design. At the same time, the project also revealed several limitations. Because the system depended on multiple subsystems running together in real time, overall reliability was affected by integration complexity, timing issues, and recognition consistency. Some features worked well individually but became harder to maintain when combined into one complete system. These limitations show that while the prototype was successful as a proof of concept, further refinement is needed before it could be considered fully robust for everyday use.

For future work, the system could be improved by making each subsystem more reliable and better optimized for real-world conditions. Future development could focus on improving gesture accuracy, making voice recognition more resistant to background noise, and strengthening the camera tracking so volume adjustments remain smooth and consistent. The interface could also be expanded with more personalization features, such as user-specific preferences, additional gesture options, and smarter mood-based music recommendations. More extensive user testing would also be valuable to evaluate usability, accessibility, and long-term performance in everyday settings. Overall, future research should build on AirWave's proof of concept by making the system more stable, scalable, and practical for

broader deployment as a next-generation music interaction platform.

VI. REFERENCES

- [1] E. Dritsas, M. Trigka, C. Troussas, and P. Mylonas, "Multimodal interaction, interfaces, and communication: A survey," *Multimodal Technologies and Interaction*, vol. 9, no. 1, p. 6, Jan. 2025, doi: 10.3390/mti9010006.
- [2] J. Summer, *Adoption of Smart Home Speakers*. Munich, Germany: GRIN Verlag, 2023.
- [3] G. C. Hillar, *MQTT Essentials: A Lightweight IoT Protocol: The Preferred IoT Publish-Subscribe Lightweight Messaging Protocol*. Birmingham, U.K.: Packt Publishing, 2017.
- [4] L. Kane, B. K. Dewangan, and T. Choudhury, *Challenges and Applications for Hand Gesture Recognition*. Hershey, PA, USA: IGI Global, 2022.
- [5] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. New York, NY, USA: Pearson, 2018.
- [6] S. Furui, *Digital Speech Processing*. Boca Raton, FL, USA: CRC Press, 2018.
- [7] D. Aniebo, "How to build real-time update systems with MQTT and Express.js," *freeCodeCamp.org*. [Online]. Available: <https://www.freecodecamp.org/news/how-to-build-real-time-update-system-s-with-mosquitto-and-expressjs/>. [Accessed: Oct. 14, 2026].
- [8] A. Jaramillo-Yáñez, M. E. Benalcázar, and E. Mena-Maldonado, "Real-time hand gesture recognition using surface electromyography and machine learning: A systematic literature review," *Sensors*, vol. 20, no. 9, p. 2467, Apr. 2020, doi: 10.3390/s20092467.
- [9] C. Schmandt, *Voice Communication with Computers*. New York, NY, USA: Van Nostrand Reinhold, 1994.
- [10] Carnegie Mellon University, "Recognizing speech recognition," *College of Engineering, Carnegie Mellon University*. [Online]. Available: <https://www.ece.cmu.edu/news-and-events/story/2020/03/stern-speech-reco-ginitation.html>. [Accessed: Nov. 21, 2026].